Lido PR85

Security Review

Cantina Managed review by:
Noah Marconi, Lead Security Researcher
Sujith Somraaj, Lead Security Researcher
Cccz, Security Researcher

March 5, 2026

Contents

1 Introduction **2**
1.1 About Cantina 2
1.2 Disclaimer 2
1.3 Risk assessment 2
1.3.1 Severity Classification 2

2 Security Review Summary **3**
2.1 Scope 3

3 Overview **4**

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Lido is the leading liquid staking solution, providing a simple way to get rewards on digital tokens.

From Mar 4th to Mar 5th the Cantina team conducted a review of the fixes implemented in PR 85 (at commit hash b5bbb129), as a follow-up of Lido's August 2023 review by Cantina. No additional vulnerabilities were identified during the review.

2.1 Scope

The security review had the following components in scope for lido-l2 on commit hash b5bbb129:

```
zksync/l1/contracts/L1ERC20Bridge.sol
```

3 Overview

Lido implemented in PR 85 a fix for the following finding:

Description: The `claimFailedDeposit` flow does not verify that the provided `l1Token` matches the supported bridge token. An attacker can submit a failed L2 deposit proof for a legitimate user deposit and pass a malicious ERC20 as `l1Token`. Because `depositAmount[_depositSender][_l2TxHash]` is keyed only by sender and transaction hash, the function reads a valid amount, deletes the mapping entry, and attempts transfer through attacker-controlled token logic. This clears the user's recovery record while leaving real bridged `wstETH` locked in the L1 bridge contract, making funds recoverable only via contract upgrade/governance intervention.

```
function claimFailedDeposit(address _depositSender, address _l1Token, bytes32 _l2TxHash,
↪  ...) external {
    uint256 amount = depositAmount[_depositSender][_l2TxHash];
    require(amount > 0, "No deposit");
    delete depositAmount[_depositSender][_l2TxHash];
    IERC20(_l1Token).safeTransfer(_depositSender, amount);
}
```

The Lido team fixed this issue by enforcing strict token validation in `claimFailedDeposit` (e.g., `require(_l1Token == supportedL1Token)`), and deriving the expected token from canonical bridge configuration rather than user input.